



LAB_04 PWM OUTPUT

Embedded Systems 3

Fontys University of Applied Science

09-10-2021

Stijlaart, Casey C.

466488

Abstract

Making use of a timer allows the programmer to run an time based process on in the background, without having to interfere with it. Regarding to a timer output PWM¹ mode there is the possibility to periodically switch the digital outputs HIGH and LOW. Therefore is this document, to cover the understanding on how PWM output timers work, the approach of how to set them up and use them.

Table of Contents

1. Introduction	2
2. Research	3
2.1. Hardware	3
2.2. Algorithm	3
2.2.1. Part A	3
2.2.2. Part B	3
2.2.3. Part C	3
2.3. MCP internal.....	3
3. Design	4
3.1. Part A	4
3.2. Part B	4
3.3. Part C	4
4. Proof of concept	5
4.1. Part A	5
4.2. Part B	5
4.3. Part C	6
5. Reflection.....	8
6. References	8

Table of figures

Figure 1 : Flowchart part A.	4
Figure 2 : Flowchart part B.	4
Figure 3 : Flowchart part C.	4
Figure 4 : Logic Analyser part A.	5
Figure 5 : Full pulsating cycle.....	5
Figure 6 : Shortest pulse part B.	6
Figure 7 : Longest pulse part B.	6
Figure 8 : Pulse -90 degrees.....	6
Figure 9 : Pulse 0 degrees.....	7
Figure 10 : Pulse 90 degrees.....	7

¹ Pulse Width Modulator

1. Introduction

The timer PWM mode makes use of the capture/compare mode, meaning that the timer will continue to count until the assigned CCRx value has been met. Once this value is reached, the corresponding output pin is assigned to the programmable mode. Regarding to this assignment, the programmable mode is the PWM mode 1. The PWM mode allows the programmer to set more specific values as an output, in comparison with the 'standard' HIGH and LOW.

Throughout this report there will be discussed how I tackled using PWM output timers regarding to this assignment. The assignment itself is split up into multiple parts, each part delivers a different problem that needs to be solved. Therefore regarding to each part, there will be discussed how their own problems have been researched and solved.

Regarding to the different parts of the assignment, in the first part, part A, a LED needs to be on at a 75% brightness. The second part, part B, challenges to make a LED pulse based on a brightness of 20% and 100%. Lastly, in part C a servo needs to rotate based on the PWM. For the exact requirements see (Andova, LAB_04_PWM_Output, 2021).

2. Research

2.1. Hardware

The hardware that is chosen depends on which part is being discussed, therefore regarding to part A and part B there is only made use of the Nucleo F303RE and its build-in LED. In the last part, part C, there is also made use of a Servo besides the Nucleo F303RE itself. Lastly, there is also made use of a logic analyser to proof my concept.

2.2. Algorithm

For each part there is made use of the same formula to calculate the values. The used formulas is as following :

$$Duty\ Cycle = \frac{CCR}{ARR} \quad ARR = period \cdot clock\ Hz - 1$$

2.2.1. Part A

The LED needs to be turned on with a 75% brightness, meaning its duty cycle should be 75% (0,75). Based on this the needed CCR1 value can be calculated. For this calculation there will be made use of an reload value of 160 000 – 1.

$$0.75 = \frac{CCR}{160000-1} \quad CRR = 0.75 \cdot (160000 - 1) \approx 119\,999$$

2.2.2. Part B

Throughout this part, the LED needs to be turned on with a 100% (1,0) and a 20% (0,2). Therefore there are two different CCR values. The first one, 100%, will be the same as the reload value itself, since the LED should be on the complete cycle. Regarding to the second one, there does need to be a CCR value calculated.

$$0.2 = \frac{CCR}{160000-1} \quad CRR = 0.2 \cdot (160000 - 1) \approx 3\,200$$

2.2.3. Part C

The used reload value for this part is different then in the previous parts. Regarding to this part there is made use of a reload value for 20ms.

$$ARR = 0.02s \cdot 16 \times 10^6 - 1 = 320\,000 - 1$$

Regarding to the CCR value, there are three different ones needed in this part. These values are based a duty cycle period. These duty cycles are as following, 1ms/20ms, 1.5ms/20ms and 2ms/20ms. Regarding to this assignment there is made used of a reload value of

$$\frac{1}{20} = \frac{CCR}{320000-1} \quad CRR = \frac{1}{20} \cdot (320000 - 1) \approx 16\,000$$

$$\frac{1.5}{20} = \frac{CCR}{320000-1} \quad CRR = \frac{1.5}{20} \cdot (320000 - 1) \approx 24\,000$$

$$\frac{2}{20} = \frac{CCR}{320000-1} \quad CRR = \frac{2}{20} \cdot (320000 - 1) \approx 32\,000$$

2.3. MCP² internal

The MCP is mainly focussed on setting up the timer in such a way that there is made use of a PWM output compare mode. Therefore, it's register bit needs to be enabled and the output pins need to be set to the correct mode. To set up the timer I followed the steps which were provided in (Andova, timers_out_v2, 2021).

² Manual Control Panel

3. Design

3.1. Part A

The LED will be on for a period of time, which is decided by its assigned CCR1 value. When this value is met by the counter, the LED will turn off until the counter has reached its reload value. The created flowchart for this situation is represented in Figure 1 : Flowchart part A.

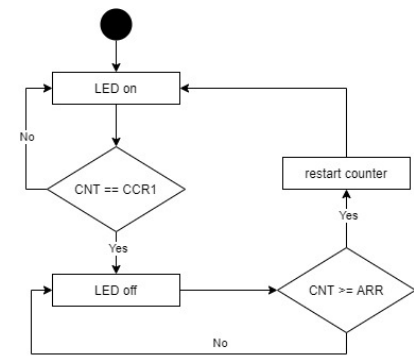


Figure 1 : Flowchart part A.

3.2. Part B

In the second part the LED has to pulsate, meaning that the CCR value will increase when it starts from the lowest pulse. Once it reaches the max pulse value, it will start to decrease again until the lowest pulse. This process will continuously repeat itself. Between every duty cycle, the counter will be reloaded once it has reached its reload value. For the final diagram, see Figure 2 : Flowchart part B.

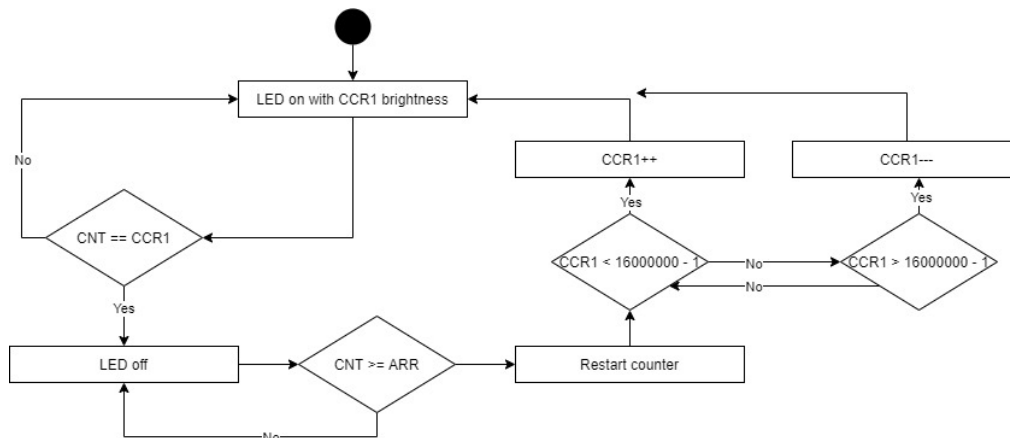


Figure 2 : Flowchart part B.

3.3. Part C

In the last part, there will be a servo rotating based on the pulse. The pulse is based on how long the occurred duty cycle was. There are three different options for the way the servo rotates. When a 1ms pulse occurs, it rotates to position -90 degrees. When it was a 1.5ms pulse, it will rotate to position 0 degrees. Lastly when it was a 2ms pulse, the servo will rotate to a 90 degrees position. After a pulse has occurred, the CCR1 value will be updated to the next expected pulse value. Regarding to the diagram, see Figure 3 : Flowchart part C.

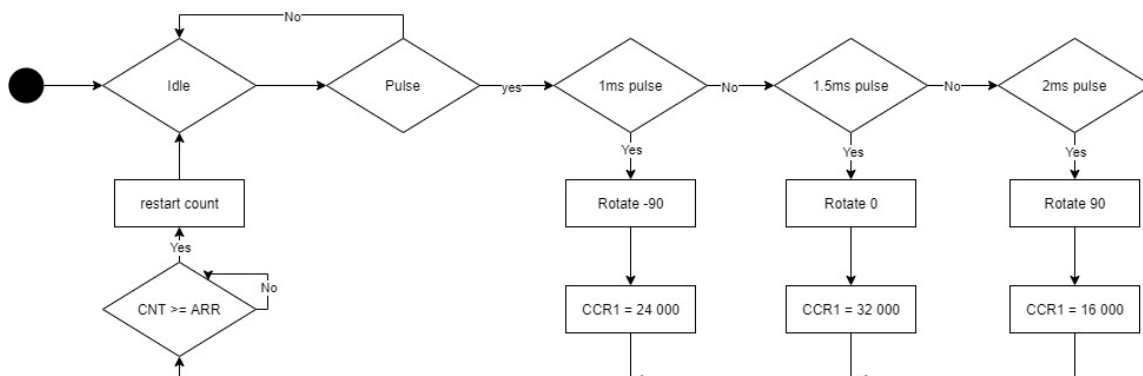


Figure 3 : Flowchart part C.

4. Proof of concept

4.1. Part A

To display my findings and how the written program works, there is made use of a logic analyser. With the help of this tool, there can be displayed how the clock is influenced by the duty cycle, and therefor there can be read how long the duty cycle is. See Figure 4 : Logic Analyser part A.

To check if the duty cycle is what has been anticipated, the found periods can be divided by each other. $Duty\ Cycle = \frac{15}{20} \approx 75\%$. This results tells that the expected duty cycle is indeed what is delivered.

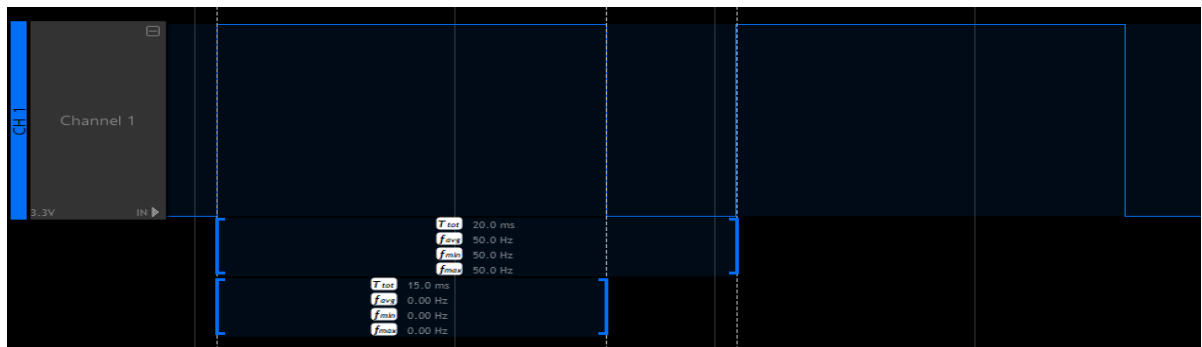


Figure 4 : Logic Analyser part A.

4.2. Part B

In order to showcase how the duty cycle increases and decreases throughout a period of time. A capture could be made with the help of a logic analyser, see Figure 5 : Full pulsating cycle. To get a more clear overview of how the pulse is manipulated overtime, the reload value has been modified to a larger value. With this there can be concluded that the duty cycle is indeed manipulated in such a way that the LED will pulsate. To make it does pulsate from a brightness of 20% to 100%, the shortest and the longest pulse will be measured. Based on these findings the duty cycle for each is calculated and conclude if it behaves the way as it's expected. For the shortest pulse, see Figure 6 : Shortest pulse part B. The $Duty\ Cycle = \frac{4.50}{20.1} \approx 22\%$. For the longest pulse, see Figure 7 : Longest pulse part B. The $Duty\ Cycle = \frac{19.6}{20} \approx 98\%$.

Both of these calculations show that they longest and shortest pulse are slightly off. This can be explained due to the fact that the CCR1 value is increased with an unspecific duty cycle. Therefor the duty cycle can vary with each full period of pulsating from 20% to 100% and back.

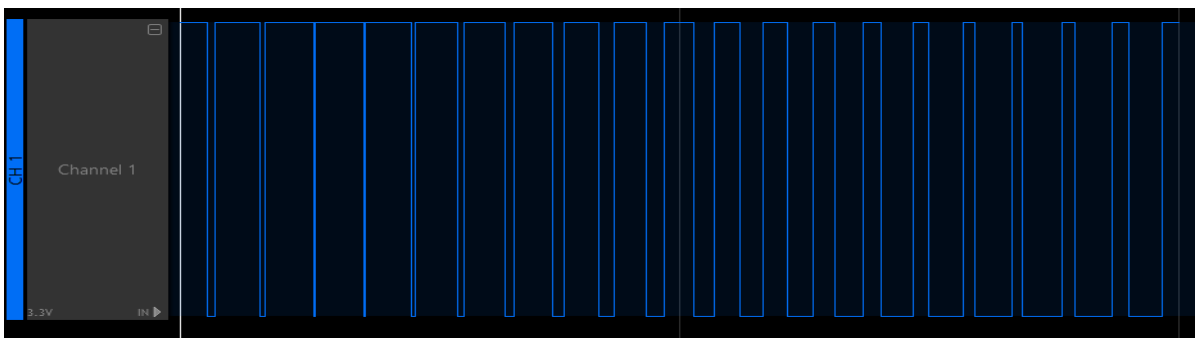


Figure 5 : Full pulsating cycle.

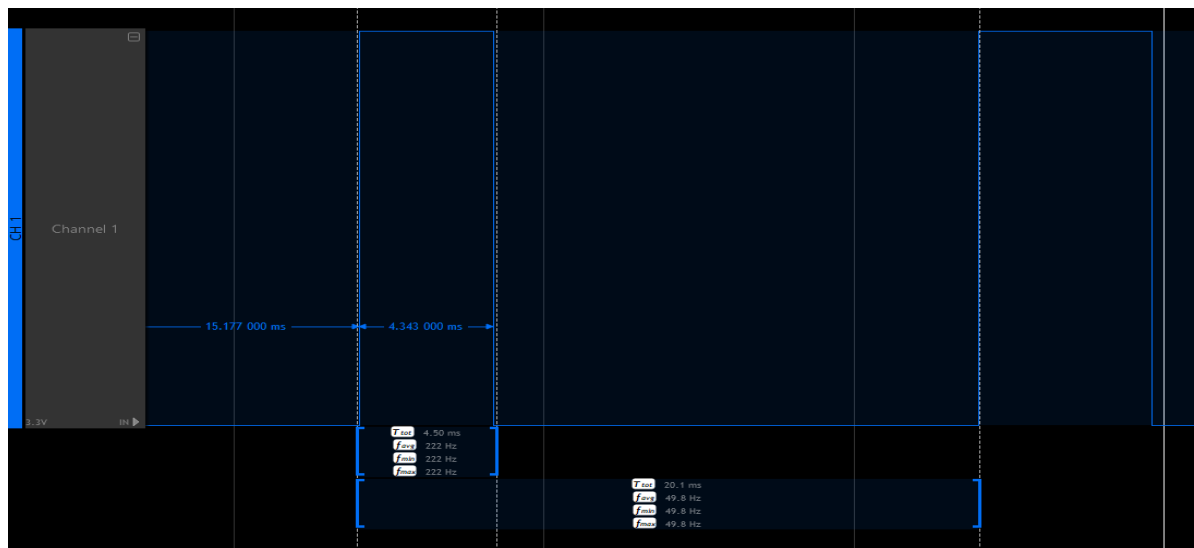


Figure 6 : Shortest pulse part B.

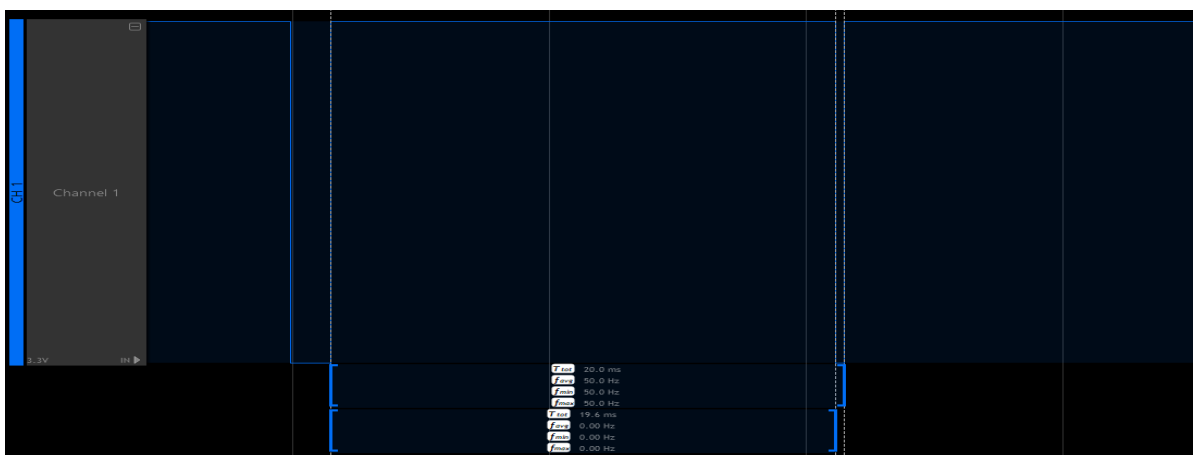


Figure 7 : Longest pulse part B.

4.3. Part C

Throughout working on this part, there had to be made adjustment to the CCRx values in order to get the desired results. The desired result was to have a 90 degrees angles between the 1ms and 1.5ms servo position and between 1.5ms and 2ms. When trying to out the values, which had been calculated in 2.2.3. Part C, the angles were noticeably of and there for had to be manually adjusted based on visuals. This caused to get different values for the short and medium pulse. As in the figures can be seen, the chosen values resulted in a different duty cycle. The used values results in the following CRRx values; -90 degrees is 10 000, 0 degrees is 24 000 and 90 degrees is 38 000. See Figure 8 : Pulse -90 degrees. Figure 9 : Pulse 0 degrees. and Figure 10 : Pulse 90 degrees.

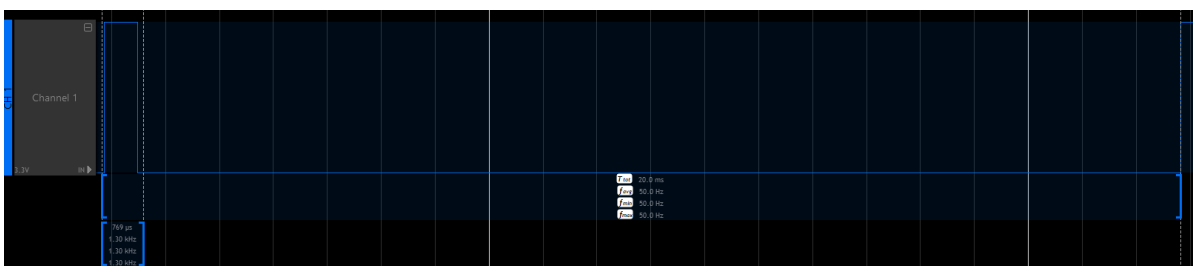


Figure 8 : Pulse -90 degrees.

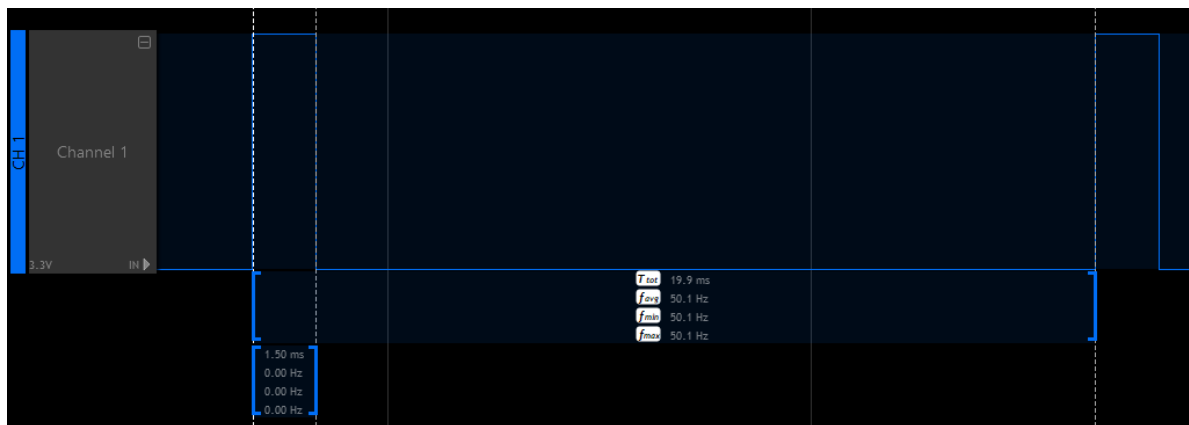


Figure 9 : Pulse 0 degrees.



Figure 10 : Pulse 90 degrees.

5. Reflection

Throughout the assignment I noticed a significant difference in the way I approached on work on each part in comparison as how I did it before. I took more time to carefully read into the problems I had to solve, and wanted to solve them. Therefor I started drawing flowcharts before actually starting on programming. This is a huge win for me regarding to personal growth. Before I would just jump into it, without actually thinking well about it.

Even though it went much smoother regarding to the starting of the process, there were quiet a few problems that got ran into while programming. The main reason for these problems were how I interpreted the assignment and did not ask for help soon enough. This meant that I got stuck at certain problems for a long period of time, and resulted in a higher stress level and pressure to perform. Once a problem would be solved, I was able to easily keep moving through the assignment. Unfortunately I was unable to solve one of the problems I ran in to. During the last part of the assignment the timer and the SysTick interfered with each other, therefor I was unable to make the servo repeatedly rotate based on the chosen CCRx value. Despite this, I am content with how I performed. I have the feeling that I keep improving my way of working, and that it truly pays of.

Regarding to what I've learned about the subject itself, I mainly learned how a timer can be configured and how it can be used to manipulate a PWM signal. Besides that, I learned how the timer can be changed in such ways that the timer interval can influence how the PWM output is displayed. For example, when looking at part B it will not actually look like it pulsates when the reload value for the timer is bigger, so a bigger interval. It will just looks like it is blinking, when making the reload value much smaller, the output will actually look like its pulsating since the blinking is so fast that the human eye can't actually see it. Therefor the eye will register it as an LED which get's brighter, less bright again and so on.

6. References

Andova, S. (2021). *LAB_04_PWM_Output*. From git.fhict.nl: https://git.fhict.nl/technology/t-sem3-cb/-/blob/master/es/Timers/Lab_04_PWM_Output.pdf

Andova, S. (2021). *timers_out_v2*. From git.fhict.nl: https://git.fhict.nl/technology/t-sem3-cb/-/blob/master/es/Timers/timers_out_v2.pptx

Brown, G. (2016, June 5). *Discovering the STM32 Microcontroller*. From legacy.cs.indiana.edu: <https://legacy.cs.indiana.edu/~geobrown/book.pdf>

Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C. (2016, december 9). *Lecture 13: Timer PWM Output*. From youtube: <https://www.youtube.com/watch?v=zkrVHicLGww>

life.augmented. (n.d.). *STM32L4 timers*. From st.com: https://www.st.com/content/ccc/resource/training/technical/product_training/c4/1b/56/83/3a/a1/47/64/STM32L4_WDG_TIMERS_GPTIM.pdf/files/STM32L4_WDG_TIMERS_GPTIM.pdf/jcr:content/translations/en.STM32L4_WDG_TIMERS_GPTIM.pdf

st. (2017, January). *DM00043574 ReferenceManual STM32F303RE*. From Git: https://git.fhict.nl/technology/t-sem3-cb/-/raw/master/es/Intro_STM32_microcontroller/DM00043574-ReferenceManual_STM32F303RE.pdf